

Test-time Sparsity for Extreme Fast Action Diffusion

Kangye Ji^{1*}, Yuan Meng^{1*,†}, Jianbo Zhou^{1*}, Ye Li¹, Chen Tang², Zhi Wang^{1†}

¹Tsinghua University ²The Chinese University of Hong Kong

Abstract

Action diffusion excels at high-fidelity action generation but incurs heavy computational costs owing to its iterative denoising nature. Despite current technologies showing promise in accelerating diffusion transformers by reusing the cached features, they struggle to adapt to policy dynamics arising from diverse perceptions and multi-round rollout iterations in open environments. We propose test-time sparsity to tackle this challenge, which aims to accelerate action diffusion by dynamically predicting prunable residual computations for each model forward at test time. However, two bottlenecks remain in this paradigm: 1) repetitive conditional encoding and pruning offset most potential speed gains, and 2) the features cached from previous denoising timesteps cannot constrain large pruning errors under aggressive sparsity. To address the first bottleneck, we design a highly parallelized inference pipeline that minimizes the non-decoder delay to milliseconds. Specifically, we first design a lightweight pruner that shares the encoder with the diffusion transformer. Then, we decouple the encoding and pruning from the autoregressive denoising loop by processing all denoising timesteps in parallel, and overlap the pruner with the decoder forward inference through asynchronism. To overcome the second bottleneck, we introduce an omnidirectional reusing strategy, which achieves 95% sparsity by selectively reusing the features cached from the current forward, previous denoising timesteps, and earlier rollout iterations. To learn the rollout-level reusing strategies, we sample a few action trajectories to supervise the sparsified diffusion step by step. Extensive experiments demonstrate that our method reduces FLOPs by 92% and accelerates action generation by 5 $\times$, achieving lossless performance with an inference frequency of 47.5 Hz. Our code is available at <https://github.com/ky-ji/Test-time-Sparsity>.

1. Introduction

Action diffusion [4, 21, 25] has garnered substantial attention in robotic control due to its remarkable ability to model multi-

*Equal contribution. Work done when Jianbo Zhou was an intern at mmlab@SIGS, Tsinghua University.

[†]Corresponding authors.

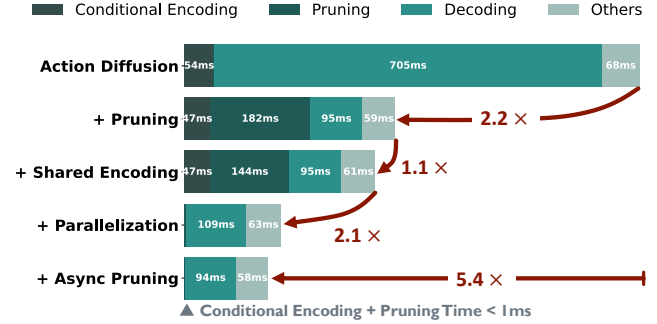


Figure 1. Inference Latency Breakdown. Test-time sparsity removes 5% of the decoder computations by omnidirectionally reusing historical features (Section 3.3), and reduces the delay of conditional encoding and pruning to within one millisecond through a highly parallelized inference pipeline (Section 3.2).

modal action distributions [4, 11, 16, 17, 29]. This capability has made it essential for action generation modules in recent visuomotor policies [4, 29] and advanced Vision-Language-Action (VLA) models [1, 7, 12, 15, 24, 27], particularly for complex and highly dexterous manipulation tasks. However, the adoption of action diffusion is fundamentally constrained by its low achievable frequency inherent in the iterative denoising process, e.g., 6Hz for Diffusion Policy [4] and 5Hz for 3D Diffusion Policy [29] on a consumer-grade GPU, far below the 30 Hz required by many real-world tasks [20].

Despite current technologies showing promise in accelerating diffusion transformers by reusing the cached features, they struggle to adapt to the evolving dynamics of visuomotor policies in open environments. A line of works [2, 6] accelerate the diffusion by reusing the previous partially denoised actions from earlier rollout iterations, while another line of studies [9, 28] reuse the intermediate features from the previous denoising steps. However, these approaches rely on static schedules, which are inherently mismatched with the dynamic nature of the environment. In other words, they fail to account for the evolving sparsity patterns arising from the multi-round interactions and diverse perceptions that emerge in open, ever-changing environments.

To address this gap, we propose test-time sparsity for extreme fast action diffusion, aiming to predict prunable resid-

ual computations before each model forward. The paradigm follows a prune-then-reuse pipeline [10], where a parameterized pruner dynamically predicts the skippable residual computations of the forward, and then the action diffusion transformer skips the sparse residuals, reducing the pruning error by reusing the features cached from previous timesteps. However, such a paradigm raises two primary inference bottlenecks that hinder its practical efficiency: 1) repetitive conditional encoding and pruning offset most speed gains (as shown in Figure 1), 2) the features cached from timesteps are insufficient to constrain large pruning errors under aggressive sparsity (refer to Table 6).

To address the first bottleneck, we design a highly parallelized inference pipeline that minimizes the non-decoder delay to milliseconds. Specifically, we first design a lightweight parameterized pruner that shares the encoder with the action diffusion transformer, which reduces a 40ms delay, as shown in Figure 1. Then, we decouple the encoding and pruning from the autoregressive denoising loop by processing all denoising timesteps in parallel at test time. Finally, the pruner is overlapped with the decoder forward inference through asynchronism, which reduces the non-decoder delay to milliseconds.

To overcome the second bottleneck, we introduce omnidirectional reusing, which achieves 95% sparsity by efficiently reusing historical features cached from multiple directions. The insight stems from the observation that features cached from different directions are not only highly aligned with the anchored ones but also offer complementary advantages such as different angles and shorter distances, as illustrated in Figure 7. To operationalize this insight, we first model the historical feature space as a 3D lattice, which efficiently organizes the cached features and provides three distinct candidates for each anchored feature. On top of this structure, we design the pruner output as a unified embedding that jointly decides whether to perform computation and which cached feature to reuse. To overcome the limitations of per-forward supervision in learning rollout-level reusing strategies, we sample the action trajectories to supervise the action generated by the sparsified diffusion step by step.

To assess the effectiveness of the proposed method, we conduct experiments on extensive benchmarks. Our approach prunes up to 95% of computations during action diffusion across the entire rollout, yielding $5\times$ wall-clock speedup and achieving a 47.5 Hz inference frequency on an NVIDIA 4090 GPU without any performance degradation. In summary, our main contributions are threefold:

1. We propose test-time sparsity for accelerating action diffusion, which dynamically prunes the diffusion transformer to accommodate evolving policy dynamics.
2. We unleash the instantiation potential of test-sparsity by two key designs: a highly parallelized inference pipeline that reduces the non-decoder overhead to milliseconds,

and an omnidirectional reusing strategy that achieves 95% sparsity by selectively reusing the features cached from the current forward, previous denoising timesteps, and earlier rollout iterations.

3. We conduct comprehensive experiments on diverse manipulation benchmarks, covering both visuomotor policies and VLA models. The results show that our method achieves $5\times$ speedup while maintaining high generation fidelity.

2. Related Work

Action Diffusion Acceleration. Current acceleration techniques for action diffusion broadly fall into (i) distillation-based model compression and (ii) reuse-based computation sparsity. Distillation-based methods learn fast surrogates that approximate the original multi-step denoising trajectory. One-Step Diffusion Policy [26] converts a pretrained policy into a single-step action generator. Consistency Policy [22] and VDD [30] distill a Diffusion Policy into a faster student policy by enforcing self-consistency along diffusion trajectories. In terms of reuse-based methods, both Falcon [2] and Streaming Policy [6] reuse partially denoised trajectories in the previous iterations. Other methods reuse the intermediate features. EfficientVLA [28] adopts a uniform reusing schedule, i.e., updating the cached features at fixed intervals. Block-wise Adaptive Caching [9] accelerates policy inference by reusing intermediate action features at the block level. However, these methods can not adjust the schedules dynamically according to policy dynamics.

Reuse-based Image Diffusion Acceleration. Previous works explored the sparsity of diffusion models well by skipping redundant activations with cached features. Early methods [14, 19] focus on U-Net backbones, while more recent approaches [3, 13, 18, 23, 32] focus on transformer architectures. Due to the diffusion models typically undergoing a one-shot generation process, these methods primarily adopt predefined strategies to reuse historical features from previous timesteps. Most of them identify and prune the most relevant features based on heuristics, such as temporal proximity [14] or feature similarity [23]. Other works design caching strategies [3] tailored to the specific patterns of the image generation process. Notably, despite some works [18, 31] learn the sparsity pattern by aligning the action before and after sparsification, they still generate the learned schedules offline to avoid the additional overhead.

3. Methodology

In this section, we first outline preliminaries in action diffusion in Section 3.1. Next, we introduce the parallelized inference pipeline in Section 3.2 and describe the omnidirectional reusing strategy in Section 3.3.

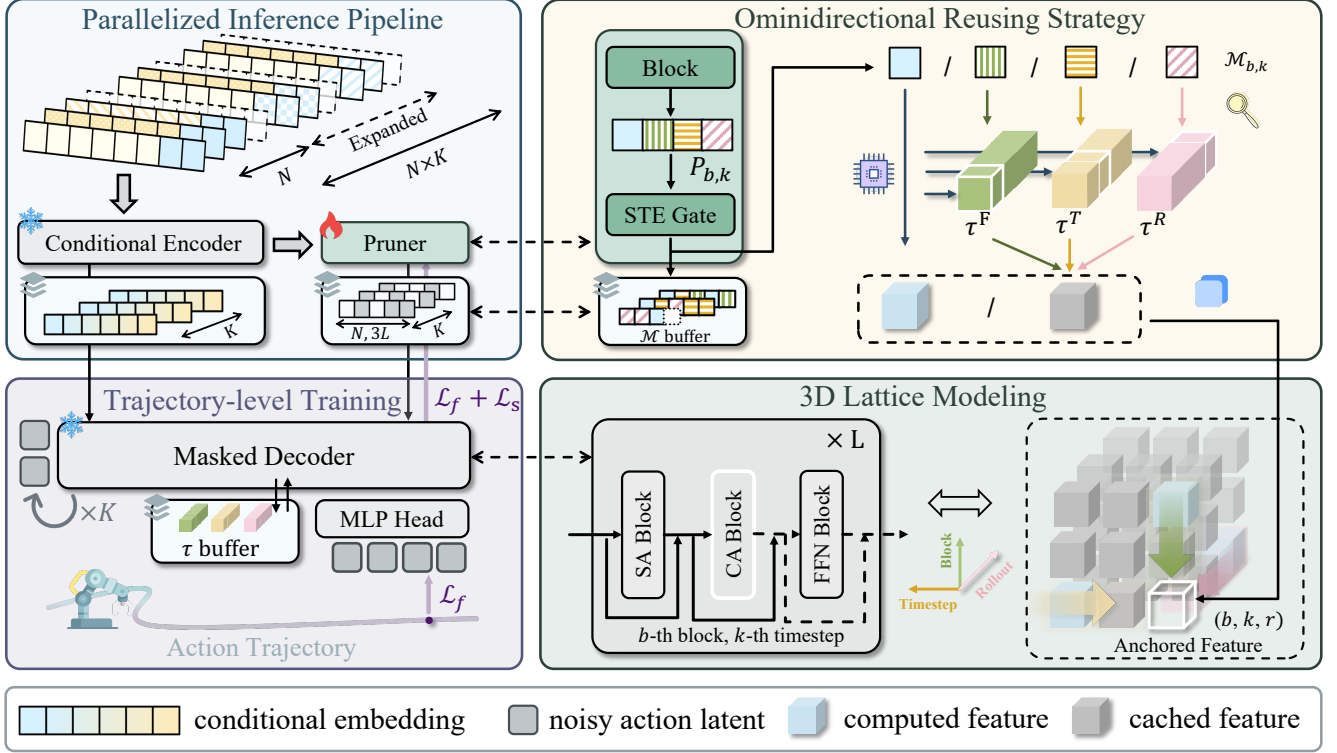


Figure 2. The overview of Test-time Sparsity. The paradigm comprises four key components: 1) Parallelized Inference Pipeline that reduces non-decoder latency to milliseconds; 2) Omnidirectional Reusing which achieves extreme sparsity by adaptively reusing features cached from the current forward, previous denoising timesteps, and earlier rollout iterations; 3) 3D Lattice Modeling for efficiently organizing large-scale historical features; and 4) Trajectory-level Training that supervises the learning of Omnidirectional Reusing strategy.

3.1. Preliminary

Action Diffusion Transformer. The action diffusion transformer consists of a conditional encoder and a transformer decoder. At each denoising timestep k , the conditional encoder maps the diffusion timestep into a sinusoidal embedding and transforms the current observation $\mathbf{o}_r$ into an embedding sequence via an MLP. These embeddings are then concatenated with learnable positional encodings to form the conditional representation. The transformer decoder takes the noisy action $\mathbf{a}_r^k$ as input tokens and refines it through L stacked layers. Each layer l comprises a self-attention (SA) block, a cross-attention (CA) block, and a feed-forward network (FFN). The conditional embeddings are injected into the decoder via the cross-attention blocks, enabling effective conditioning across timesteps. Given the hidden state $\mathbf{h}_k^{l-1}$ at denoising step k , the update is a residual summation:

$$\mathbf{h}_k^l = \mathbf{h}_k^{l-1} + \text{SA}_k^l + \text{CA}_k^l + \text{FFN}_k^l. \quad (1)$$

The output of layer l is computed by summing the residual outputs of these blocks. As the block is the minimal residual computation unit, it serves as the pruning target in this work.

Action Diffusion. The action diffusion describes the visuomotor generation process in a diffusion manner during the policy-environment interaction. At each iteration r , the policy π receives the current observation $\mathbf{o}_r$ and produces an action chunk $\mathbf{a}_r$, which is executed by the robot. The environment then evolves under its dynamics and yields the next observation $\mathbf{o}_{r+1}$:

$$\mathbf{o}_{r+1} \sim P(\mathbf{o}_{r+1} | \mathbf{o}_r, \mathbf{a}_r), \quad (2)$$

where $P(\cdot | \mathbf{o}_r, \mathbf{a}_r)$ denotes the environment’s state transition probability. This process repeats for multiple iterations.

The action $\mathbf{a}_r$ used in each iteration is obtained via conditional denoising diffusion [4]. Starting from Gaussian noise $\mathbf{a}_r^K \sim \mathcal{N}(0, I)$, the policy performs K reverse denoising steps using the denoiser f_θ :

$$\mathbf{a}_r^{k-1} = f_\theta(\mathbf{a}_r^k, \mathbf{o}_r, k), \quad k = K, \dots, 1. \quad (3)$$

We can therefore formulate the entire multi-step denoising process, which maps initial noise to a final action:

$$\mathbf{a}_r = \pi(\mathbf{o}_r) = \mathbf{a}_r^0 = f_\theta \circ f_\theta \circ \dots \circ f_\theta(\mathbf{a}_r^K, \mathbf{o}_r). \quad (4)$$

The final $\mathbf{a}_r$ serves as the control action in Eq. 2.

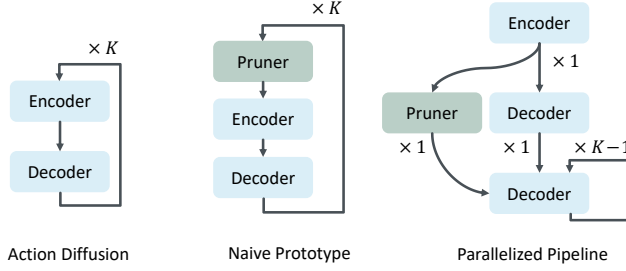


Figure 3. Comparison of different inference pipelines. The parallelized pipeline runs the pruner and encoder only once, and overlaps the runtime of the pruner with the initial decoder step.

3.2. Parallelized Inference Pipeline

Prototype of Test-time Sparsity. To adapt to changing dynamics of visuomotor policies in an open environment, we necessitate a pruner that identifies the skippable residual blocks for each forward inference adaptively. We condition the pruner with environment perceptions $\mathbf{o}_r$ and the current denoising timestep k , which causally govern the sparsity pattern of each forward process. The pruner is further expected to output a binary pruning mask $\mathcal{M} \in \{0, 1\}^{3L}$ for all $3L$ DiT blocks. The prototype is illustrated in Figure 3.

Decoupled Pruner by Parallelization. The primary inference bottleneck in the prototype arises from repetitive operations within the autoregressive denoising loop, including decoding, conditional encoding, and pruning. As shown in Figure 1, pruning at test-time has addressed the repetitive decoding well by skipping and reusing previously cached features, which reduces the corresponding delay from 705ms to 95ms in each diffusion. However, this solution introduces an additional significant overhead: the pruner itself requires 182ms for its own repetitive inference, a cost that even surpasses the 95ms decoding time achieved at 95% sparsity.

To reduce the significant inference overhead of the pruner, a naive idea is to decouple the pruner from the iterative loop by encoding all the timesteps in one go. However, as evidenced by the higher curve in Figure 5, the pruner performs substantially worse under all-step encoding, revealing a fundamental trade-off between the efficiency of all-step encoding and the accuracy attainable with single-step encoding.

To address this dilemma, we parallelize the prediction of the pruner by fully exploiting the GPU at inference time. Although the pruner model is designed to process a single timestep k , we reformulate this as a large-scale batched operation at inference. Specifically, we first compute the sinusoidal positional embeddings for all K denoising timesteps simultaneously. We then collapse the temporal dimension K into the data batch dimension N , creating a single, large batch with an effective size of $N \times K$, as illustrated in Fig-

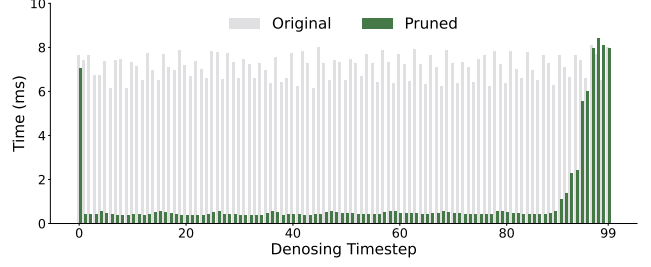


Figure 4. Inference time breakdown during denoising. The inference time after pruning is mainly concentrated in the later steps.

ure 2. This unified tensor is fed into the pruner in a single forward pass, generating pruning masks for all timesteps concurrently. This parallel formulation transforms the K -step sequential loop into a fully-batched operation, thereby eliminating most of the pruner-induced delay.

Lightweight Pruner Design. We instantiate the pruner with a lightweight transformer decoder block. To further reduce the latency of conditional encoding, we share the condition encoder between the pruner and the action diffusion transformer, enabling the pruner to reuse high-level condition embeddings without incurring additional cost. Moreover, by encoding all timesteps in parallel, the conditional encoder not only significantly reduces inference delay but also seamlessly supports the parallelized pruner.

Concretely, the pruner first encodes all the block indices using sinusoidal positional embeddings to obtain block-specific query embeddings. The pruner then processes these block embeddings as the target sequence within its transformer decoder block, which is conditioned on the output of the shared condition encoder from the action diffusion transformer. The decoder outputs are subsequently passed through an MLP head to produce the pruning mask $\mathcal{M}$.

Asynchronous Pipeline. To facilitate efficient coordination within the decoupled paradigm, we design an asynchronous pipeline. Specifically, two buffers are introduced for the conditional encoder and the pruner to store the condition embeddings and pruning masks generated at once. Afterward, the pipeline transitions into a decoder-only loop, where the decoder directly retrieves these cached features for conditioning and masking, thereby eliminating sequential dependencies and waiting time.

We further analyze the inference time during a single diffusion process and observe in Figure 4 that the pruner skips nearly all redundant computations in the early steps, except for the first step, due to the necessary full forward for cache initialization. To hide the runtime of the pruner, we launch two parallel threads for the pruner and decoder after conditional encoding, as illustrated in Figure 3, enabling

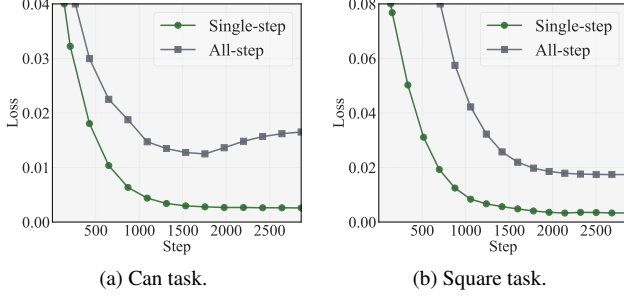


Figure 5. Loss convergence with different timestep encodings.

only 0.45ms overhead for the pruning at test time.

3.3. Ominidirectional Reusing Strategy

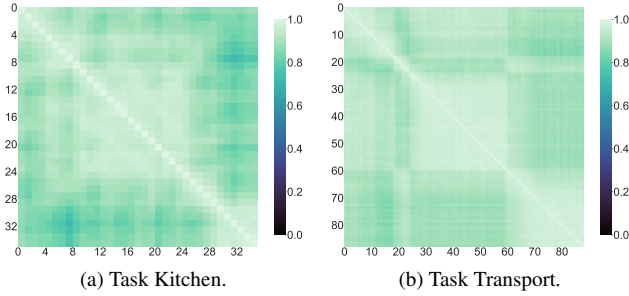
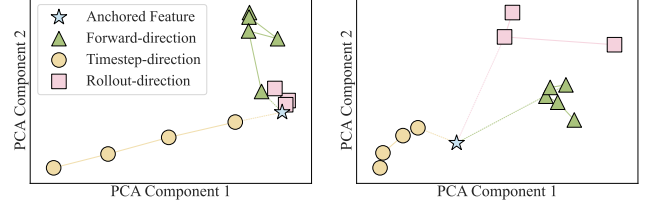


Figure 6. Similarity between features from different rollouts.

Motivation. While prior caching techniques reuse features from previous timesteps [9, 19, 28], this single-direction paradigm yields sub-optimal performance under high sparsity (See Table 6). Critically, we first observe that features from different rollout iterations exhibit strong similarities, as shown in Figure 6. Furthermore, by anchoring one feature and visualizing the cached features from various rollout directions, we find that these features are well aligned with the anchored one, each contributing complementary advantages in approaching angles and latent distances, as illustrated in Figure 7. These observations uncover a significant yet previously underexplored potential for feature reuse. Motivated by this, we envision an efficient reusing strategy that efficiently utilizes the massive historical features, thereby presenting two critical challenges: 1) How to efficiently organize the massive features during action diffusion (e.g., 50k-200k for Diffusion Policy [4])? 2) How to select the most suitable one from them for reuse?

3D Lattice Modeling. To address the first challenge, we first model the historical feature space as a 3D lattice defined by three orthogonal indices: block index b , denoising timestep k , and rollout iteration r . Each anchored feature exists at a coordinate (b, k, r) in this space. Our omnidirectional reusing strategy organizes the cache by maintaining



(a) Anchored feature from the 6th self-attention block at the 100th denoising step during the 30th rollout step on the Tool task. (b) Anchored feature from the 4th self-attention block at the 95th denoising step during the 30th rollout step on the Tool task.

Figure 7. PCA visualization of features cached from different directions, i.e., the current forward, previous denoising timesteps, and earlier rollout iterations, which are closely aligned with the anchored ones, and offering complementary advantages.

the closest available feature along each of these three axes. As illustrated in Figure 2, a cached feature differs from its anchored counterpart by only one index. For example, a feature cached from a previous rollout has coordinates $(b, k, r - 1)$, sharing the same block and timestep indices. Globally, we maintain separate, dedicated buffers for all three directions, retaining only the most recently updated feature along each axis. This structure efficiently assigns three candidates for each anchored feature.

Trajectory-level Training. To address the second challenge, we incorporate the cache-selection mechanism directly into the pruner. For the b -th block at denoising timestep k , the pruner outputs a 4-dimensional gating vector

$$\mathbf{p}_{b,k} = (p_{b,k}^C, p_{b,k}^F, p_{b,k}^T, p_{b,k}^R), \quad (5)$$

where each element represents the confidence in either performing a new computation or reusing a feature cached from the forward, denoising timestep, or rollout directions. During inference, the gating vector is discretized via an $\arg \max$ operation to yield the mask vector $\mathcal{M}_{b,k}$. To enable gradient-based optimization, we adopt the Straight-Through Estimator [8] during training, which allows the gradient to flow through the non-differentiable $\arg \max$ operation while maintaining discrete gating behavior in the forward pass. The residual update is thus formulated as

$$h_k^{[b/3]} = h_k^{[b/3]-1} + \mathcal{M}_{b,k}^C d_{b,k} + \mathcal{M}_{b,k}^F \tau_b^F + \mathcal{M}_{b,k}^T \tau_b^T + \mathcal{M}_{b,k}^R \tau_{b,k}^R. \quad (6)$$

where $d_{b,k}$ denotes the newly computed feature, and τ_b^F, τ_b^T , and $\tau_{b,k}^R$ are features cached from different directions. The corresponding caches are refreshed according to:

$$\tau_b^F \leftarrow (1 - \mathcal{M}_{b,k}^C) \tau_b^F + \mathcal{M}_{b,k}^C d_{b,k} \quad (7)$$

$$\tau_b^T \leftarrow (1 - \mathcal{M}_{b,k}^C) \tau_b^T + \mathcal{M}_{b,k}^C d_{b,k} \quad (8)$$

$$\tau_{b,k}^R \leftarrow (1 - \mathcal{M}_{b,k}^C) \tau_{b,k}^R + \mathcal{M}_{b,k}^C d_{b,k} \quad (9)$$

Table 1. **Benchmark on Proficient Human (PH) demonstration data.** Our method achieves $5.18\times$ speedup with lossless performance under 95% sparsity. **Best** and **second best** performances are highlighted.

Method	Sparsity(%)	Success Rate (% , $\uparrow$) / Speedup ($\uparrow$)					Average	GFLOPS($\downarrow$)
		Lift	Can	Square	Transport	Tool		
<i>Dense</i>	0	100	94	90	80	50	83	7.88
EfficientVLA	86	100 (3.49 $\times$)	74 (3.39 $\times$)	90 (3.53 $\times$)	60 (3.29 $\times$)	38 (3.57 $\times$)	72 (3.46 $\times$)	1.24
L2C	26	100 (1.48 $\times$)	86 (1.27 $\times$)	26 (1.31 $\times$)	66 (1.32 $\times$)	2 (1.33 $\times$)	56 (1.28 $\times$)	5.87
BAC	90	100 (3.66 $\times$)	94 (3.78 $\times$)	94 (3.79 $\times$)	84 (3.43 $\times$)	26 (3.74 $\times$)	79 (3.68 $\times$)	1.07
Ours	93	100 (4.90 $\times$)	94 (4.96 $\times$)	90 (4.85 $\times$)	92 (4.70 $\times$)	56 (4.88 $\times$)	86 (4.86 $\times$)	0.68
Ours	95	100 (5.31 $\times$)	88 (5.39 $\times$)	92 (5.18 $\times$)	94 (4.83 $\times$)	48 (5.21 $\times$)	84 (5.18 $\times$)	0.42

To learn the reuse strategy along the rollout direction, we sample a few action trajectories and supervise the sparsified diffusion outputs step by step across the rollout iterations. During each trajectory, the pruner predicts a binary mask $\mathcal{M}^r$ in each rollout iteration r , and the action diffusion sparsified by $\mathcal{M}^r$ outputs an action $\hat{a}_r$. After each diffusion step, gradients are backpropagated, thus enabling multi-step supervision throughout the entire trajectory. In each iteration, the pruner is jointly optimized with a fidelity loss $\mathcal{L}_f$ and a sparsity regularization loss $\mathcal{L}_s$:

$$\mathcal{L} = \mathcal{L}_f + \mathcal{L}_s, \quad (10)$$

where

$$\mathcal{L}_f = \mathbb{E}_{(o_r, a_r^*) \sim \mathcal{D}_{\text{ref}}} [\|\pi'(\mathbf{o}_r, \mathcal{M}^r) - a_r^*\|], \quad (11)$$

$$\mathcal{L}_s = \left| \frac{1}{BK} \sum_{b=1}^B \sum_{k=1}^K p_{b,k}^c - (1 - \rho) \right|. \quad (12)$$

Here, a_r^* is the reference action at rollout iteration r , and ρ represents the target pruning rate.

4. Experiment

We first outline the experimental setup in Section 4.1. Following that, we demonstrate the quantitative results in Section 4.2. We conduct an ablation study of the method in Section 4.3, and present qualitative analysis in Section 4.4.

4.1. Experimental Setup

Models and Benchmarks. We evaluate the acceleration performance of our method on two types of action diffusion models: diffusion policy and vision-language-action (VLA) model, using the most representative architectures from each category. For diffusion policy, we adopt the transformer-based Diffusion Policy [4] as our primary evaluation backbone. Our method is tested across a range of robot manipulation tasks with fixed seeds, including Lift, Can, Square, Transport, Tool Hang, and Kitchen [5]. The demonstration

datasets consist of trajectories collected from proficient human (PH) and mixed proficient/non-proficient human (MH) teleoperation. To further assess the generality of our approach under VLA settings, we extend our experiments to the state-of-the-art RDT-1B model [16], which incorporates a 1-billion-parameter diffusion transformer for action generation, coupled with T5-XXL for language conditioning and SigLIP for visual encoding. We evaluate our method on four ManiSkill simulation tasks: PushCube, PickCube, StackCube, and Insert.

Metrics. We assess the performance of different methods using the *Success Rate* over 50 episodes, following prior works [4]. We evaluate the efficiency by reporting *Sparsity*, *Speedup*, and *GFLOPS*. *Sparsity* indicates the percentage of pruned computations during inference. *Speedup* measures the wall-clock time reduction compared to the full-precision model. *GFLOPS* quantifies the total floating-point operations required for each action diffusion.

Baselines. We establish the original models as our *Dense* baseline. For comparison, we benchmark against leading acceleration methods. We reproduce current cache-based acceleration methods for action generation, Efficient-VLA [28] and BAC [9]. Additionally, we adapt L2C [18], a learning-based acceleration method originally designed for image generation, for our action generation tasks.

Implementation Details. The experiments are conducted on NVIDIA Tesla A40 48G GPU, equipped with Intel(R) Xeon(R) Gold 6230 CPU@2.10GHz. The pruner introduces a hyperparameter pruning rate ρ , which we set as 80%, 90%, 93%, and 95% for different settings. The pruner network is trained for 20 epochs with a learning rate of $1e^{-4}$ and weight decay of $1e^{-4}$. We set the batch size as 16 and 1 for Diffusion Policy and RDT-1B, respectively.

4.2. Quantitative Results

For Diffusion Policy, we test two sampling configurations: DDPM (100 denoising steps) and DDIM (40 denoising steps). For RDT-1B, we follow the original paper and adopt DPM-Solver with 50 steps. For L2C, we adopt the same

Table 2. **Benchmark on Mixed Human (MH) demonstration data.** Our method achieves over $4.6\times$ speedup across all tasks. **Best** and **second best** performances are highlighted.

Method	Sparsity(%)	Success Rate (% , $\uparrow$) / Speedup ($\uparrow$)				Average	GFLOPS($\downarrow$)
		Lift	Can	Square	Transport		
<i>Dense</i>	0	100	94	74	56	81	7.88
EfficientVLA	86	100 (3.45 $\times$)	76 (3.31 $\times$)	52 (3.37 $\times$)	0 (3.69 $\times$)	57 (3.46 $\times$)	1.24
L2C	26	100 (1.28 $\times$)	0 (1.33 $\times$)	54 (1.25 $\times$)	46 (1.33 $\times$)	50 (1.30 $\times$)	5.87
BAC	90	100 (3.69 $\times$)	70 (3.73 $\times$)	78 (3.72 $\times$)	50 (3.65 $\times$)	74 (3.70 $\times$)	1.06
Ours	93	100 (4.86 $\times$)	94 (4.94 $\times$)	82 (4.83 $\times$)	62 (4.69 $\times$)	85 (4.83 $\times$)	0.72
Ours	95	100 (5.30 $\times$)	94 (5.30 $\times$)	74 (5.24 $\times$)	50 (5.02 $\times$)	80 (5.22 $\times$)	0.44

Table 3. **Benchmark on multi-stage task (Kitchen).** In the Kitchen task, p_x is the frequency of interacting with x or more objects, including an openable microwave, four turnable oven burners, an oven light switch, and a freely movable kettle. Our method achieves a lossless $5.90\times$ speedup on this task. **Best** and **second best** performances are highlighted.

Method	Sparsity(%)	Success Rate (% , $\uparrow$)				Average	Speedup ($\uparrow$)	GFLOPS($\downarrow$)
		Kit _{p1}	Kit _{p2}	Kit _{p3}	Kit _{p4}			
<i>Dense</i>	0	100	100	100	100	100	–	113
EfficientVLA	86	20	2	0	0	3	3.60 $\times$	13.81
L2C	25	100	100	100	98	99	1.28 $\times$	85.5
BAC	90	100	98	94	82	93	3.90 $\times$	15.83
Ours	93	100	100	100	100	100	5.90 $\times$	9.71
Ours	95	100	100	98	98	99	6.33 $\times$	7.28

training configs as our method for fair comparison. We set the cache updating interval $\mathcal{N}$ as 7 for EfficientVLA. We set the number of cache update steps $\mathcal{S}$ as 8 and the number of selected upstream blocks k as 5 for BAC. We conduct quantitative evaluations as shown in Tables 1, 2, 3, 4 and 5. **Effectiveness on Diffusion Policy.** As shown in Tables 1, our method prunes over 95% computations during action diffusion while achieving lossless performance on PH dataset, demonstrating great sparsity of action diffusion. Moreover, Tables 1, 2, and 3 report that our method achieves comparable performance with the original model under 93% sparsity across all tasks, reaching up to $6.33\times$ speedup on the challenging multi-stage Kitchen task.

Collaboration with Efficient Sampling Methods. As shown in Tables 4 and 5, our method also demonstrates effectiveness when combined with efficient samplers like DDIM and DPM-Solver. On the Diffusion Policy with DDIM sampler using 40 denoising steps, our method achieves a lossless performance with over $3\times$ speedup under 80% sparsity across all tasks. For RDT-1B with DPM-Solver using 50 denoising steps, our method attains over $2.5\times$ speedup under 90% sparsity while maintaining or even improving the success rates on all ManiSkill tasks.

Effectiveness on VLA. We report the acceleration results on the large-scale RDT-1B model in Table 5. Under 90%

Table 4. **Acceleration on DDIM.** We evaluate Diffusion Policy with a DDIM sampler using 40 denoising steps on MH datasets under 80% sparsity. **Best** performances are highlighted.

Method	Success Rate (% , $\uparrow$) / Speedup ($\uparrow$)			
	Kitchen	Lift	Square	Transport
<i>Dense</i>	98	100	78	52
Ours	100 (3.64 $\times$)	100 (2.96 $\times$)	82 (3.06 $\times$)	56 (2.87 $\times$)

sparsity, our method achieves significant speedups of over $2.5\times$ across all ManiSkill tasks while maintaining or even improving the success rates compared to the original model. We observe the lower speedup compared to Diffusion Policy, mainly due to the heavier visual and language encoders in RDT-1B, which limit the end-to-end acceleration.

Comparison with SOTA baselines. Our method shows significant advantages over existing state-of-the-art acceleration methods, which fail to deliver comparable performance under high sparsity. L2C [18] forces computing the features before each reuse step, resulting in a modest $1.28\times$ speedup. EfficientVLA follows a naive reusing strategy that reuses in a fixed interval, which leads to poor performance, such as a 3% success rate on the Kitchen task. Despite BAC achiev-

Table 5. **Quantitative Results on VLA.** We evaluate our approach on the large-scale RDT-1B model across ManiSkill manipulation tasks under 90% sparsity. **Best** results are highlighted.

Method	Success Rate (% , $\uparrow$) / Speedup ($\uparrow$)			
	Push	Pick	Stack	Insert
<i>Dense</i>	100	76	80	4
Ours	100 (3.11 $\times$)	80 (2.64 $\times$)	80 (2.49 $\times$)	16 (2.67 $\times$)

ing fine-grained block-wise caching, it cannot adapt to the evolving rollout dynamics, leading to a sub-optimal trade-off between performance and efficiency. These baseline methods are limited to reusing features cached solely along the denoising-step dimension, which significantly restricts the achievable speedup. In contrast, our approach constrains the prune-induced errors successfully by the omnidirectional reusing strategy that jointly reuses features from massive historical features, which preserves the high fidelity of the generated action under even 95% sparsity. The method further transfers the advantage into $5\times$ practical speedup by the parallelized inference pipeline.

4.3. Ablation Study

To verify the effectiveness of Omnidirectional Reusing, we conduct an ablation study by reusing the features from only a single specific direction. We evaluate the variants on the PH dataset under 93% sparsity. As summarized in Table 6, all the single-direction variants suffer noticeable drops in success rate across all tasks. Moreover, each variant encounters severe performance degradation on certain tasks, such as Forward-direction on Transport, Timestep-direction on Tool, and Rollout-direction on Can, indicating that reusing features along only one direction is insufficient to compensate for the aggressive pruning. In contrast, the full omnidirectional caching scheme consistently achieves the best trade-off between performance and efficiency by selectively reusing cached features from all directions.

Table 6. **Ablation Study.** We report the success rate when only reusing features from single directions on PH dataset under 93% sparsity. **Best** performances are highlighted.

Method	Success Rate (% , $\uparrow$)			
	Can	Transport	Tool	Square
<i>Dense</i>	94	80	50	90
Forward-direction	86	4	50	18
Timestep-direction	86	78	0	80
Rollout-direction	10	70	32	80
Omini-direction	94	92	56	90

4.4. Qualitative Analysis

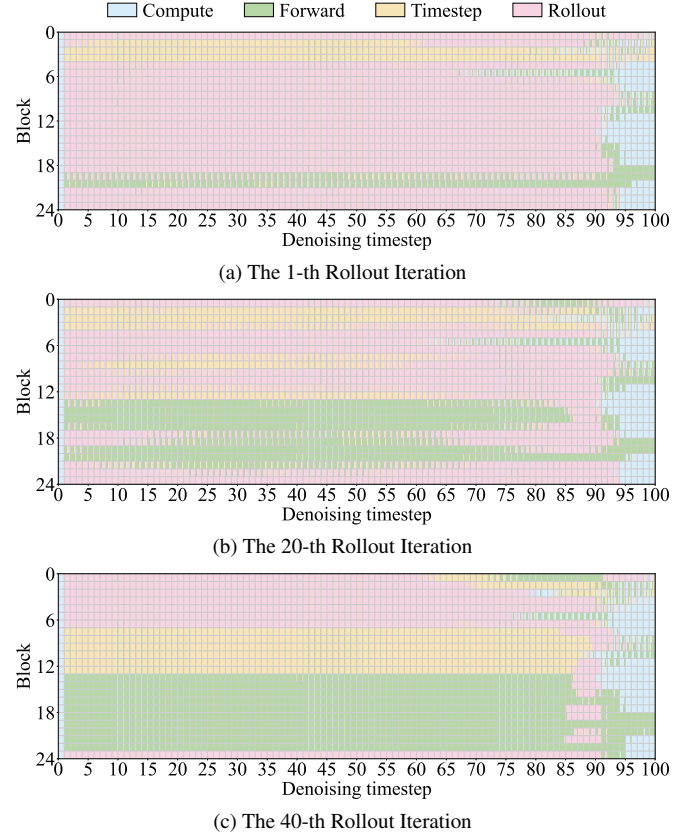


Figure 8. **Visualization of $\mathcal{M}$ during the robot lifting a small box.** Each cell at coordinate (k, b) denotes $\mathcal{M}_{b,k}$. The proportions of the colored regions within each cell indicate the confidence scores $p_{b,k}^C$, $p_{b,k}^F$, $p_{b,k}^T$, and $p_{b,k}^R$.

Figure 8 visualizes the predicted pruning mask $\mathcal{M}$. The mask $\mathcal{M}$ exhibits significant variations throughout the rollout iterations, revealing the presence of test-time sparsity that emerges alongside the evolving visuomotor dynamics in an open environment. Moreover, each direction occupies considerable proportions, highlighting the necessity of aggregating historical features from all directions. More details are provided in the supplementary material.

5. Conclusion

In this work, we propose test-time sparsity, which accelerates the action diffusion by $5\times$, achieving 47.5Hz inference frequency without performance degradation. The result benefits from two key designs: First, we design a highly parallelized inference pipeline that minimizes the non-decoder delay to milliseconds. Second, we introduce omnidirectional reusing, which achieves 95% sparsity by selectively reusing the historical features cached from the current forward, previous denoising timesteps, and earlier rollout iterations.

6. Acknowledgement

This work is supported by the National Natural Science Foundation of China (Grant No. 92467204, 62472249, and 62402264), and the Shenzhen Science and Technology Program (Grant No. JCYJ20220818101014030 and KJZD20240903102300001).

References

- [1] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. *pi.0*: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [2] Haojun Chen, Minghao Liu, Chengdong Ma, Xiaojian Ma, Zailin Ma, Huimin Wu, Yuanpei Chen, Yifan Zhong, Mingzhi Wang, Qing Li, et al. Falcon: Fast visuomotor policies via partial denoising. *arXiv preprint arXiv:2503.00339*, 2025.
- [3] Pengtao Chen, Mingzhu Shen, Peng Ye, Jianjian Cao, Chongjun Tu, Christos-Savvas Bouganis, Yiren Zhao, and Tao Chen. δ -dit: A training-free acceleration method tailored for diffusion transformers. *arXiv preprint arXiv:2406.01125*, 2024.
- [4] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44 (10-11):1684–1704, 2025.
- [5] Abhishek Gupta, Vikash Kumar, Corey Lynch, Sergey Levine, and Karol Hausman. Relay policy learning: Solving long-horizon tasks via imitation and reinforcement learning. In *Conference on Robot Learning*, pages 1025–1037. PMLR, 2020.
- [6] Sigmund H Høeg, Yilun Du, and Olav Egeland. Streaming diffusion policy: Fast policy synthesis with variable noise diffusion models. *arXiv preprint arXiv:2406.04806*, 2024.
- [7] Zhi Hou, Tianyi Zhang, Yuwen Xiong, Haonan Duan, Hengjun Pu, Ronglei Tong, Chengyang Zhao, Xizhou Zhu, Yu Qiao, Jifeng Dai, et al. Dita: Scaling diffusion transformer for generalist vision-language-action policy. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 7686–7697, 2025.
- [8] Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with gumbel-softmax. *arXiv preprint arXiv:1611.01144*, 2016.
- [9] Kangye Ji, Yuan Meng, Hanyun Cui, Ye Li, Shengjia Hua, Lei Chen, and Zhi Wang. Block-wise adaptive caching for accelerating diffusion policy. *arXiv preprint arXiv:2506.13456*, 2025.
- [10] Kangye Ji, Yuan Meng, Zhou Jianbo, Ye Li, Hanyun Cui, and Zhi Wang. Sparse actiongen: Accelerating diffusion policy with real-time pruning. *arXiv preprint arXiv:2601.12894*, 2026.
- [11] Ye Li, Jiahe Feng, Yuan Meng, Kangye Ji, Chen Tang, Xinwan Wen, Shutao Xia, Zhi Wang, and Wenwu Zhu. Ts-dp: Reinforcement speculative decoding for temporal adaptive diffusion policy acceleration. *arXiv preprint arXiv:2512.15773*, 2025.
- [12] Ye Li, Yuan Meng, Zewen Sun, Kangye Ji, Chen Tang, Jiajun Fan, Xinzhu Ma, Shutao Xia, Zhi Wang, and Wenwu Zhu. Sp-vla: A joint model scheduling and token pruning approach for vla model acceleration. *arXiv preprint arXiv:2506.12723*, 2025.
- [13] Dong Liu, Yanxuan Yu, Jiayi Zhang, Yifan Li, Ben Lengerich, and Ying Nian Wu. Fastcache: Fast caching for diffusion transformer through learnable linear approximation. *arXiv preprint arXiv:2505.20353*, 2025.
- [14] Feng Liu, Shiwei Zhang, Xiaofeng Wang, Yujie Wei, Haonan Qiu, Yuzhong Zhao, Yingya Zhang, Qixiang Ye, and Fang Wan. Timestep embedding tells: It’s time to cache for video diffusion model. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 7353–7363, 2025.
- [15] Jiaming Liu, Hao Chen, Pengju An, Zhuoyang Liu, Renrui Zhang, Chenyang Gu, Xiaoqi Li, Ziyu Guo, Sixiang Chen, Mengzhen Liu, et al. Hybridvla: Collaborative diffusion and autoregression in a unified vision-language-action model. *arXiv preprint arXiv:2503.10631*, 2025.
- [16] Songming Liu, Lingxuan Wu, Bangguo Li, Hengkai Tan, Huayu Chen, Zhengyi Wang, Ke Xu, Hang Su, and Jun Zhu. Rdt-1b: a diffusion foundation model for bimanual manipulation. *arXiv preprint arXiv:2410.07864*, 2024.
- [17] Yijun Liu, Yuwei Liu, Yuan Meng, Jieheng Zhang, Yuwei Zhou, Ye Li, Jiacheng Jiang, Kangye Ji, Shijia Ge, Zhi Wang, et al. Spatial policy: Guiding visuomotor robotic manipulation with spatial-aware modeling and reasoning. *arXiv preprint arXiv:2508.15874*, 2025.
- [18] Xinyin Ma, Gongfan Fang, Michael Bi Mi, and Xinchao Wang. Learning-to-cache: Accelerating diffusion transformer via layer caching. *Advances in Neural Information Processing Systems*, 37:133282–133304, 2024.
- [19] Xinyin Ma, Gongfan Fang, and Xinchao Wang. Deepcache: Accelerating diffusion models for free. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 15762–15772, 2024.
- [20] Yunchao Ma, Yizhuang Zhou, Yunhuan Yang, Tiancai Wang, and Haoqiang Fan. Running vlcs at real-time speed, 2025.
- [21] Tim Pearce, Tabish Rashid, Anssi Kanervisto, Dave Bignell, Mingfei Sun, Raluca Georgescu, Sergio Valcarcel Macua, Shan Zheng Tan, Ida Momennejad, Katja Hofmann, et al. Imitating human behaviour with diffusion models. *arXiv preprint arXiv:2301.10677*, 2023.
- [22] Aaditya Prasad, Kevin Lin, Jimmy Wu, Linqi Zhou, and Jeannette Bohg. Consistency policy: Accelerated visuomotor policies via consistency distillation. *arXiv preprint arXiv:2405.07503*, 2024.
- [23] Pratheba Selvaraju, Tianyu Ding, Tianyi Chen, Ilya Zharkov, and Luming Liang. Fora: Fast-forward caching in diffusion transformer acceleration. *arXiv preprint arXiv:2407.01425*, 2024.
- [24] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, et al. Smolvla: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025.

- [25] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, et al. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.
- [26] Zhendong Wang, Zhaoshuo Li, Ajay Mandlekar, Zhenjia Xu, Jiaojiao Fan, Yashraj Narang, Linxi Fan, Yuke Zhu, Yogesh Balaji, Mingyuan Zhou, et al. One-step diffusion policy: Fast visuomotor policies via diffusion distillation. *arXiv preprint arXiv:2410.21257*, 2024.
- [27] Junjie Wen, Yichen Zhu, Jinming Li, Zhibin Tang, Chaomin Shen, and Feifei Feng. Dexvla: Vision-language model with plug-in diffusion expert for general robot control. *arXiv preprint arXiv:2502.05855*, 2025.
- [28] Yantai Yang, Yuhao Wang, Zichen Wen, Luo Zhongwei, Chang Zou, Zhipeng Zhang, Chuan Wen, and Linfeng Zhang. Efficientvla: Training-free acceleration and compression for vision-language-action models. *arXiv preprint arXiv:2506.10100*, 2025.
- [29] Yanjie Ze, Gu Zhang, Kangning Zhang, Chenyuan Hu, Muhan Wang, and Huazhe Xu. 3d diffusion policy: Generalizable visuomotor policy learning via simple 3d representations. *arXiv preprint arXiv:2403.03954*, 2024.
- [30] Hongyi Zhou, Denis Blessing, Ge Li, Onur Celik, Xiaogang Jia, Gerhard Neumann, and Rudolf Lioutikov. Variational distillation of diffusion policies into mixture of experts. *Advances in Neural Information Processing Systems*, 37:12739–12766, 2024.
- [31] Haowei Zhu, Dehua Tang, Ji Liu, Mingjie Lu, Jintu Zheng, Jinzhang Peng, Dong Li, Yu Wang, Fan Jiang, Lu Tian, et al. Dip-go: A diffusion pruner via few-step gradient optimization. *Advances in Neural Information Processing Systems*, 37:92581–92604, 2024.
- [32] Chang Zou, Xuyang Liu, Ting Liu, Siteng Huang, and Linfeng Zhang. Accelerating diffusion transformers with token-wise feature caching. In *International Conference on Learning Representations (Poster Track)*, 2025.

Test-time Sparsity for Extreme Fast Action Diffusion

Supplementary Material

A. Details on Rollout Similarity

Figure 9 provides additional details on rollout-level feature similarity across the Lift, Square, Tool, and Can tasks. We observe that features from different rollout iterations exhibit consistently high similarity on all tasks, reinforcing our insight that cached features from multiple directions, particularly those from rollout iterations, can be effectively leveraged to constrain large pruning errors under aggressive pruning rates.

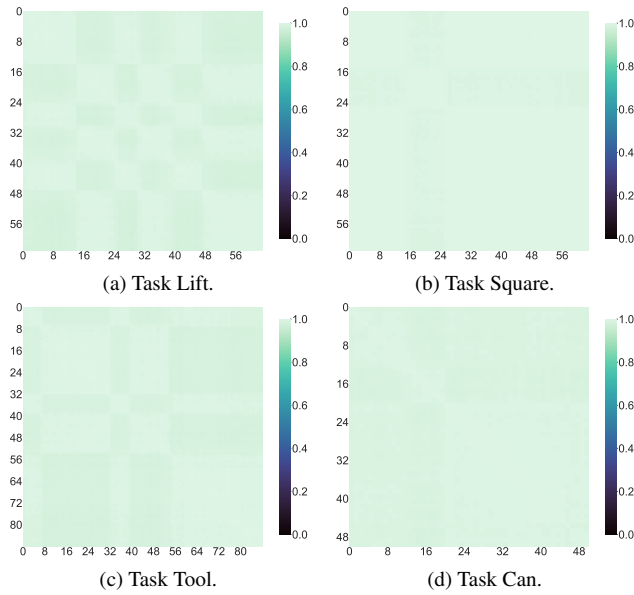


Figure 9. Similarity between features from different rollouts on Lift, Square, Tool, and Can tasks.

B. Details on Ablation Study

For a more comprehensive evaluation, we further include ablation studies on additional tasks, namely Kitchen, Lift_{ph}, and the MH dataset. The results in Tables 7 and 8 echo the findings from the main paper: single-direction reuse strategies consistently fail to maintain high success rates under aggressive sparsity across all tasks. In contrast, the proposed omnidirectional caching strategy consistently achieves the best trade-off between performance and efficiency across both tasks and datasets.

C. Training Efficiency

We assess training efficiency by reporting the success rate achieved with varying numbers of sampled trajectories, as

Method	Success Rate (% , $\uparrow$)			
	Can	Transport	Lift	Square
<i>Dense</i>	94	56	100	74
Forward-direction	64	0	2	32
Timestep-direction	72	48	98	66
Rollout-direction	82	0	98	48
Omini-direction	94	62	100	82

Table 7. **Ablation Study.** We report the success rate when only reusing features from single directions on MH dataset under 93% sparsity. **Best** performances are highlighted.

Method	Success Rate (% , $\uparrow$)				
	Kit _{p1}	Kit _{p2}	Kit _{p3}	Kit _{p4}	Lift
<i>Dense</i>	100	100	100	100	100
Forward-direction	64	0	2	32	6
Timestep-direction	72	48	98	66	4
Rollout-direction	82	0	98	48	8
Omini-direction	100	100	100	100	100

Table 8. **Ablation Study.** We report the success rate when only reusing features from single directions on Kitchen and Lift_{PH} under 93% sparsity. **Best** performances are highlighted.

shown in Figures 10 and 11. Remarkably, our method reaches performance comparable to the original model under 93% sparsity using only two trajectories, and its performance continues to improve as more trajectories are provided. These results highlight the strong data efficiency of our training pipeline.

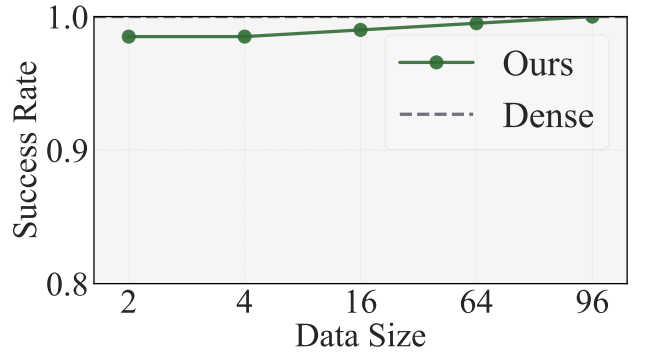


Figure 10. Success rates on the Kitchen task with different numbers of training trajectories.

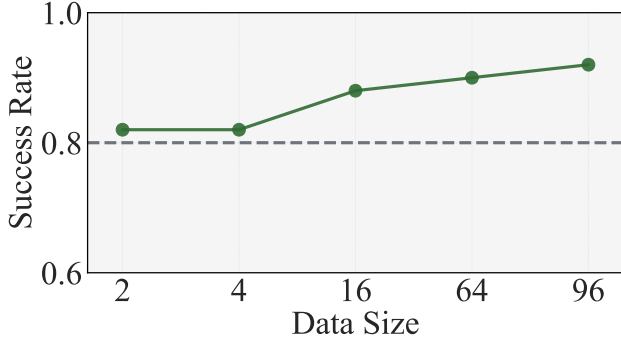


Figure 11. Success rates on the Transport task with different numbers of training trajectories.

D. Details on Qualitative Results

We present the predicted pruning masks $\mathcal{M}$ for different tasks in Figures 12, 13, 14, 15, and 16. These visualizations highlight several key observations.

First, the pruning masks differ significantly between tasks. For example, features cached from the forward direction are prioritized in the Kitchen task, whereas features from the rollout direction dominate in the Square task. This verifies that model dynamics evolve based on distinct perceptions in an open environment.

Second, a dominant trend across all tasks is that features cached from the rollout iteration, colored in pink, account for the vast majority of preserved computations. Conversely, the previously adopted temporal-level cached features, colored in yellow, account for a minimal portion. This observation strongly suggests that the rollout direction provides a more effective caching strategy for action diffusion.

Finally, the mask $\mathcal{M}$ exhibits significant variations throughout the rollout iterations, revealing evolving visuomotor dynamics within multi-round interactions. We observe other interesting phenomena, such as computations being heavily concentrated in the final steps of the rollout. We hope these observations inspire future research into efficient visuomotor model dynamics.

E. Generalization to Image Generation

Our design is tailored to the distinct characteristics of action diffusion, specifically the heavy visuomotor conditioning costs and the unexploited redundancy in long-horizon rollouts. However, the underlying principles apply broadly. The Parallelized Inference Pipeline is compatible with standard DiT-style backbones used in image/video diffusion. Moreover, Omnidirectional Reusing Strategy naturally extends to autoregressive video models (e.g., *Rolling Forcing*). It can also adapt to generic image/video generation via a variant that selectively utilizes block and timestep dimensions.

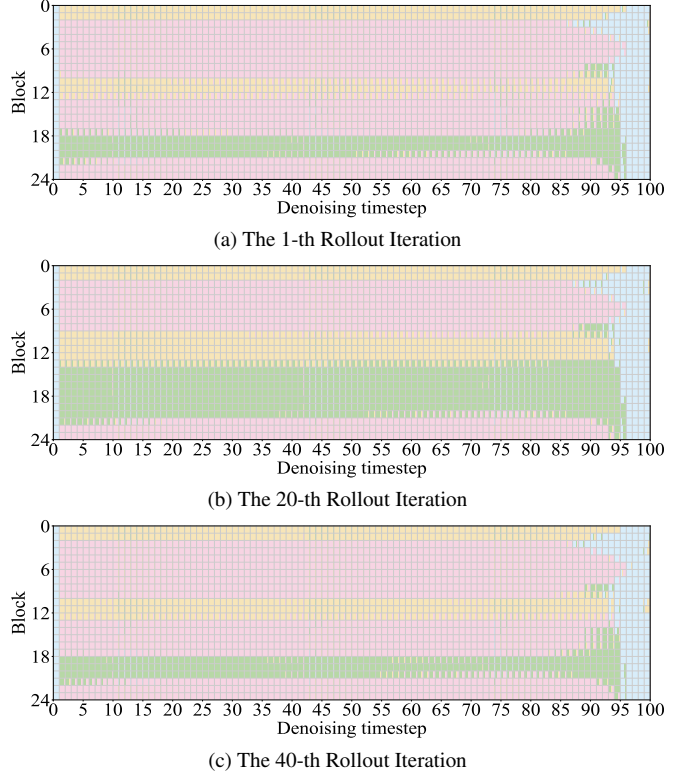
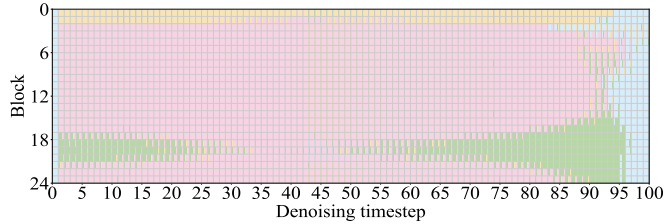


Figure 12. Visualization of $\mathcal{M}$ on Can.

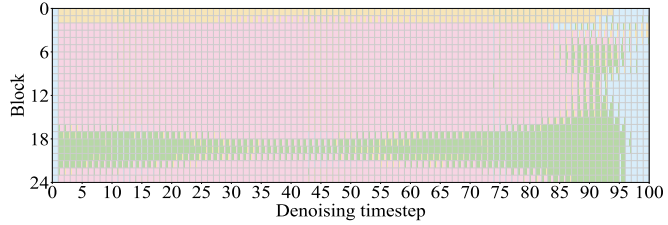
Table 9 demonstrates this strong generalization, showing negligible degradation (FID 2.23) at 80% sparsity.

Table 9. Class-to-image generation on ImageNet with DiT-XL/2.

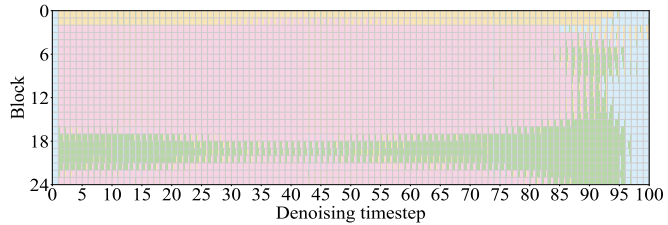
Method	Sparsity (%)	FID(↓)	sFID(↓)	Speedup(↑)
DDIM 50 steps	0	2.18	4.21	1.0×
FORA ($\mathcal{N} = 5$)	80	6.58	11.29	2.87×
ToCa ($\mathcal{N} = 6$)	83	6.55	7.10	2.63×
DuCa ($\mathcal{N} = 5$)	80	6.06	6.72	2.78×
TaylorSeer ($\mathcal{N} = 5$)	80	2.65	5.36	2.38×
Ours ($\rho = 80\%$)	80	2.23	4.37	2.63×



(a) The 1-th Rollout Iteration

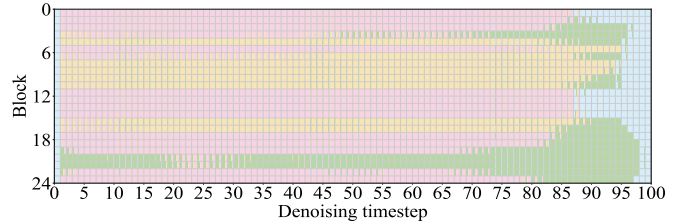


(b) The 20-th Rollout Iteration

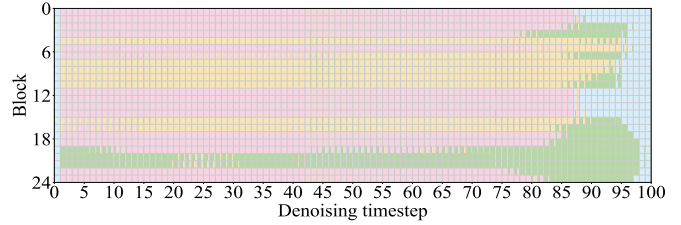


(c) The 40-th Rollout Iteration

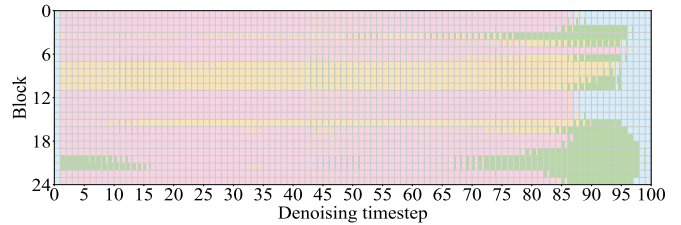
Figure 13. Visualization of $\mathcal{M}$ on Square.



(a) The 1-th Rollout Iteration

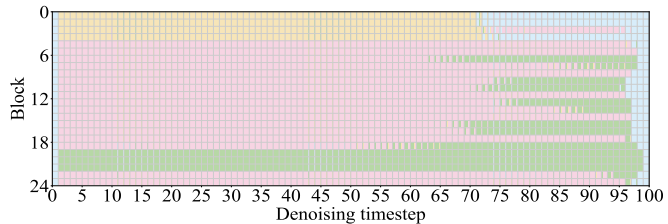


(b) The 20-th Rollout Iteration

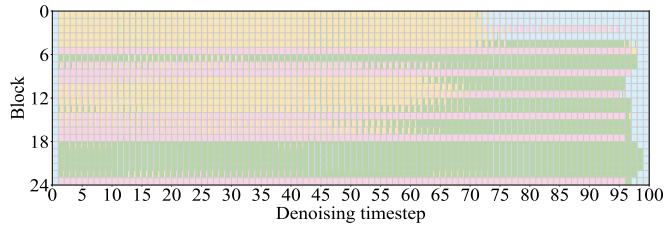


(c) The 40-th Rollout Iteration

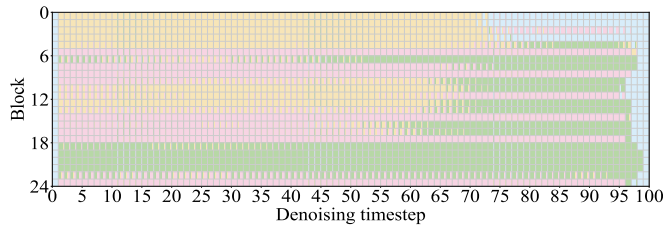
Figure 15. Visualization of $\mathcal{M}$ on Transport.



(a) The 1-th Rollout Iteration

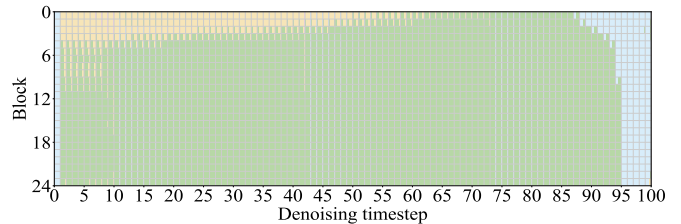


(b) The 20-th Rollout Iteration

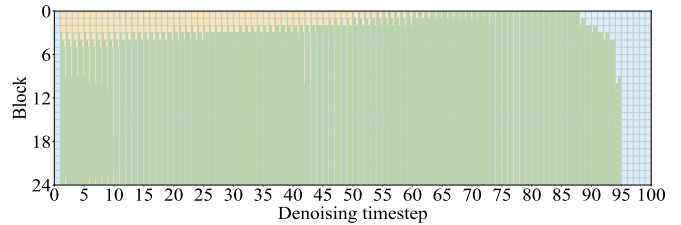


(c) The 40-th Rollout Iteration

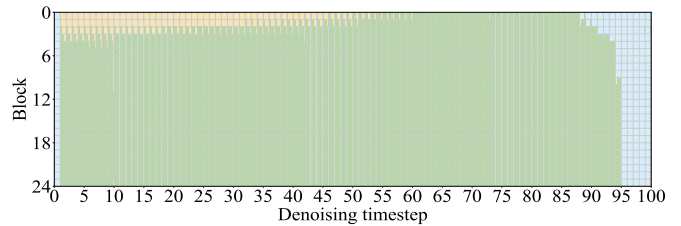
Figure 14. Visualization of $\mathcal{M}$ on Tool.



(a) The 1-th Rollout Iteration



(b) The 20-th Rollout Iteration



(c) The 40-th Rollout Iteration

Figure 16. Visualization of $\mathcal{M}$ on Kitchen.